R.M.K
GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K
GROUP OF
INSTITUTIONS

22IT958 **AGENTIC AI**

Department: **IT**

Batch/Year: **2023-2027/VI SEM**

Created by:

Dr K Saravanan/Professor

Ms.J.Divya/AP

R.M.K GROUP OF INSTITUTIONS

Table of Contents

S.NO	Topic	Page No.
1.	Course Objectives	6
2.	Pre-Requisites	7
3.	Syllabus	8
4.	Course outcomes	9
5.	CO- PO/PSO Mapping	10
6.	**Unit -II** Lecture Plan	12
7.	Activity based learning	13
8.	Lecture notes	14-49
9.	Assignments	50
10.	Part A Q & A	51-55
11.	Part B Qs	55
12.	Supportive online Certification courses	56
13.	Real time Applications in day to day life and to Industry	57
14.	Contents beyond the Syllabus	

R.M.K GROUP OF INSTITUTIONS

Course Objectives

- To introduce the basics of Agentic AI, including autonomous agents, multi-agent systems, and ethics.
- To help students learn LLM capabilities and use prompting methods like ReACT, CoT, and Plan-Execute.
- To enable students to design AI systems using LangChain, LangGraph, and Agentic AI patterns.
- To give hands-on practice in building real AI applications such as chatbots, multimodal systems, and Agentic RAG solutions.
- To make students familiar with low-code and no-code AI platforms for quick agent development and deployment.

R.M.K GROUP OF INSTITUTIONS

Syllabus

22IT958 **AGENTIC AI** **L T P C**

2 0 2 3

UNIT I INTRODUCTION TO AGENTIC AI 6+6

Agentic AI Introduction - AI Agents vs. Agentic AI -Comparison: Agentic AI, Generative AI, and Traditional AI -Agentic AI Building Blocks - Autonomous Agents, Human in the Loops Systems

Single and Multi Agent AI Systems - Agentic AI Frameworks Overview, Ethical and Responsible AI, Agentic AI Best Practices

AI Implementation Success Stories: Text generation, Translation, Summarization, Question answering, and Code generation, Reasoning, Sentiment Analysis, Classification, Translation, Image and Audio capabilities

LLM Introduction – types of LLM – LLM Capabilities - Prompt Engineering – Introduction – Elements of Prompting Engineering – Different types of Prompts – ReACT, CoT, Self Ask, Plan & Execute.

List of Exercise:

1. Use any LLM (ChatGPT / Gemini / Claude) to answer the same question with different prompting styles and compare responses.
2. Create a Python script (or notebook) that uses an LLM to do:

- Text summarization
- Sentiment analysis
- Question answering
- Students must log inputs, outputs, and observe how prompts affect results.

UNIT II AGENTIC AI MODULES & DESIGN PATTERNS 6+6

Agentic AI System Modules- Perception Module, Cognitive Module, Action Module, Learning Module, Collaboration Module, Security Module, Agentic AI Design Patterns-Reflection Pattern, Tool Use Pattern, Planning Pattern, ReAct (Reasoning and Acting) and ReWOO (Reasoning with Open Ontology),Multi Agent Pattern, Design Considerations, Model Context Protocol.

List of Excersise:

Exercise 1: Implement the ReAct Pattern

Task:

Use a simple program (LangChain optional) to demonstrate:

- The LLM deciding whether to "think" or "act"
- Take an action using a mock tool (calculator, search, etc.)
- Produce a final answer

Exercise 2: Design an Agent Architecture Diagram

Students prepare a block diagram showing:

- Perception, Cognitive, Action, Learning, Collaboration, Security modules

Task: Explain how data flows between modules for an AI task (ex: travel planning, medical triage bot, college enquiry assistant).

TOTAL :60 PERIODS

R.M.K GROUP OF INSTITUTIONS

Syllabus

22IT958 **AGENTIC AI** **L T P C**
2 0 2 3

UNIT III LANGCHAIN & LANGGRAPH FOR AGENT DEVELOPMENT 6+6

Langchain – Prompts, Chain, Tools and Agents, Memory, Runnables, LCEL, Integration with LLM Service Providers (OpenAI, Google Gemini, Ollama, Hugging Face, Claude). LangGraph – Introduction to LangGraph, State and Memory, State Schema, State Reducer, Multiple Schemas, Trim and Filter Messages, Memory and External Memory. UX and Human-in-the-Loop (HITL), Building Agent with LangGraph, Long Term Memory, Short vs. Long Term Memory, Memory Schema, Deployment.

List of Exercise:

1. Generate Text Using an LLM
2. Create a Simple Prompt Template
3. Build a LangChain agent that:

(i) Reads a table schema
(ii) Converts natural language to SQL

4. Create a Simple LangGraph Workflow
5. Create a Chatbot With Memory

UNIT IV AGENTIC RAG 6+6

What is Agentic RAG?, Agentic RAG vs. Traditional RAG,Agentic RAG Architecture and Components,Variants of Agentic RAG,Applications of Agentic RAG,Agentic RAG with Llamaindex, Agentic RAG with Cohere

List of Exercise:

1. Create a Personalized Study Material Generator using Agentic RAG
2. Generate a Student Attendance & Academic Progress Query System using Agentic RAG

UNIT V AGENTIC AI FRAMEWORKS 6+6

CrewAI – Introduction, Architecture, Task Orchestration, Agent Communication, Integration, Prompt Engineering in CrewAI, Security and Trust - AutoGen – Introduction, Core, Agent Design and Behaviour, Agent Communication,Task Executor, Workflows

Model Context Protocol implementation- Introduction to LC/NC Platforms - n8n - langflow

List of Exercise:.

1. Create two simple agents (Writer + Reviewer) and a Crew that generates and improves a paragraph.
2. Build a Crew that answers questions from a PDF (syllabus, notes, textbook chapter).
3. Create a Crew that: (i) Searches online for a topic. (ii) Summarizes the result into 5 bullet points

TOTAL :60 PERIODS

R.M.K GROUP OF INSTITUTIONS

COURSE OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Explain the basics of Agentic AI and how it differs from Traditional and Generative AI

CO2: Apply simple prompt engineering methods like ReAct (Reason + Act), CoT, Self-Ask, and Plan-Execute to improve AI agent performance.

CO3: Design AI workflows using LangChain modules such as prompts, chains, memory, tools, and agents.

CO4: Build AI agents using LangGraph with concepts like state, memory, reducers, and Human-in-the-Loop steps. effects.

CO5: Create multi-agent and Agentic RAG applications using tools like LlamaIndex, Cohere, CrewAI, and AutoGen.

CO6: Develop and deploy AI solutions using low-code and no-code platforms like n8n, Copilot Studio, Bedrock, and Vertex AI.

.

TEXTBOOKS:

1. 1. Anthony Chang, Designing Agentic AI Systems: Foundations, LLMs, and Multi-Agent Frameworks, Packt Publishing 2024.
2. 2. Richard M. Reese, Building LLM Applications with LangChain and Generative AI, O'Reilly Media, 2024.

REFERENCES:

1. 1. Thuyan Nguyen & Carlos Fernandez, Practical Multi-Agent Systems with CrewAI and AutoGen, Apress, 2024
2. 2. Nachiappan, AI Research TeamComputer Generative AI with Python and LLMs, 6th edition, Morgan Kaufmann, 2021.
3. 3. Mark Roy & Google Cloud Experts, Building AI Applications with Amazon Bedrock & Vertex AI, Wiley,Edition: 2024

SOFTWARE REQUIREMENTS:

Python

PyCharm IDE

AutoGen and CrewAI

LCNC Platforms – CrewAI / CoPilot Studio (Subscription required)

Subscriptions to any LLM Providers like OpenAI or Gemini.

R.M.K GROUP OF INSTITUTIONS

Course outcomes

CO#	COs	K-Level
CO1	Explain the basics of Agentic AI and how it differs from Traditional and Generative AI	K2
CO2	Apply simple prompt engineering methods like ReAct (Reason + Act), CoT, Self-Ask, and Plan-Execute to improve AI agent performance	K3
CO3	Design AI workflows using LangChain modules such as prompts, chains, memory, tools, and agents	K3
CO4	Build AI agents using LangGraph with concepts like state, memory, reducers, and Human-in-the-Loop steps.	K4
CO5	Create multi-agent and Agentic RAG applications using tools like LlamaIndex, Cohere, CrewAI, and AutoGen.	K4
CO6	Develop and deploy AI solutions using low-code and no-code platforms like n8n, Copilot Studio, Bedrock, and Vertex AI.	

R.M.K GROUP OF INSTITUTIONS

CO- PO/PSO Mapping

CO#	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CO1						2	3				3				3
CO2					1	2	3				3				3
CO3						2	3				3				3
CO4					1	2	3				3				3
CO5					1	2	3				3				3
CO6			1	1	1										3

R.M.K GROUP OF INSTITUTIONS

UNIT - IV

Agentic RAG

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Lecture Plan

S. No	Proposed Lecture Topic	Actual Lecture Period	Pertaining CO(s)	Highest Cognitive Level	Mode of Delivery	Delivery Resources	After successful completion students should be able to (LO Outcomes)	Remarks
1	Introduction to Agentic AI and Agents		CO2	K2	MD2	T1	Understand concept of Agentic AI and autonomous agents	
2	What is Agentic RAG?		CO2	K2	MD2	T1	Explain working of Agentic Retrieval Augmented Generation	
3	Agentic RAG vs Traditional RAG		CO2	K2	MD2	T1	Compare traditional RAG and Agentic RAG approaches	
4	Agentic RAG Architecture and Components		CO2	K2	MD2	T1	Describe architecture, agents, retriever, memory and tools	
5	Variants of Agentic RAG (Single Agent, Multi Agent, Orchestration)		CO2	K2	MD2	T1	Understand different architectural variants of Agentic RAG	
6	Applications of Agentic RAG		CO2	K2	MD2	T1	Identify real-world use cases in industry and research	
7	Agentic RAG with LlamaIndex		CO2	K2	MD2	T1	Understand how LlamaIndex supports data retrieval for agents	
8	Agentic RAG with Cohere		CO2	K2	MD2	T1	Explain usage of Cohere models in Agentic RAG systems	

R.M.K GROUP OF INSTITUTIONS

UNIT – IV AGENTIC RAG

Agentic RAG (Agentic Retrieval Augmented Generation) is an advanced AI architecture where an **AI agent intelligently retrieves, reasons, plans, and uses external knowledge** before generating the final response.
Unlike normal RAG, the system does not just retrieve data — it **acts like an intelligent decision-making agent**.

What is Agentic RAG?

Agentic RAG is an advanced version of Retrieval-Augmented Generation (RAG) where an AI agent retrieves external information and autonomously decides how to use that data. In traditional RAG, the system retrieves information and generates output in one continuous process but Agentic RAG introduces autonomous decision-making.
Let's see what Agentic AI and Agents are and how they enhance RAG:
Agentic AI: Agentic AI refers to artificial intelligence systems that can perform autonomous decision-making. These agents don't follow a fixed set of instructions rather they learn from the environment and adapt their actions based on real-time data.
Agents: Agents are autonomous entities that make decisions based on their understanding of the world and the data available to them. These agents can retrieve information, process it and generate actions based on what they have learned. What sets agents apart is their ability to not just react but to actively engage with their environment.

Agentic RAG vs. Traditional RAG

Feature	Traditional RAG	Agentic RAG
Decision-Making	Reactive, no autonomous decisions. It follows predefined workflows.	Proactive, autonomously decides what to retrieve and how to act.
Data Retrieval	Uses fixed, predefined sources like documents and databases.	Dynamically retrieves from multiple, diverse external sources.
Flexibility	Low flexibility; static retrieval and generation methods.	High flexibility; adapts retrieval and processing strategies
Adaptability	Limited adaptability; struggles with new or dynamic inputs.	Highly adaptable; continuously refines and improves performance.
Autonomy	Dependent on explicit user queries; no self-initiated action.	Operates independently, learns and adapts in real-time.
Use Case	Suitable for FAQs, simple Q&A and static search.	Ideal for dynamic chatbots, recommendation systems and complex workflows.

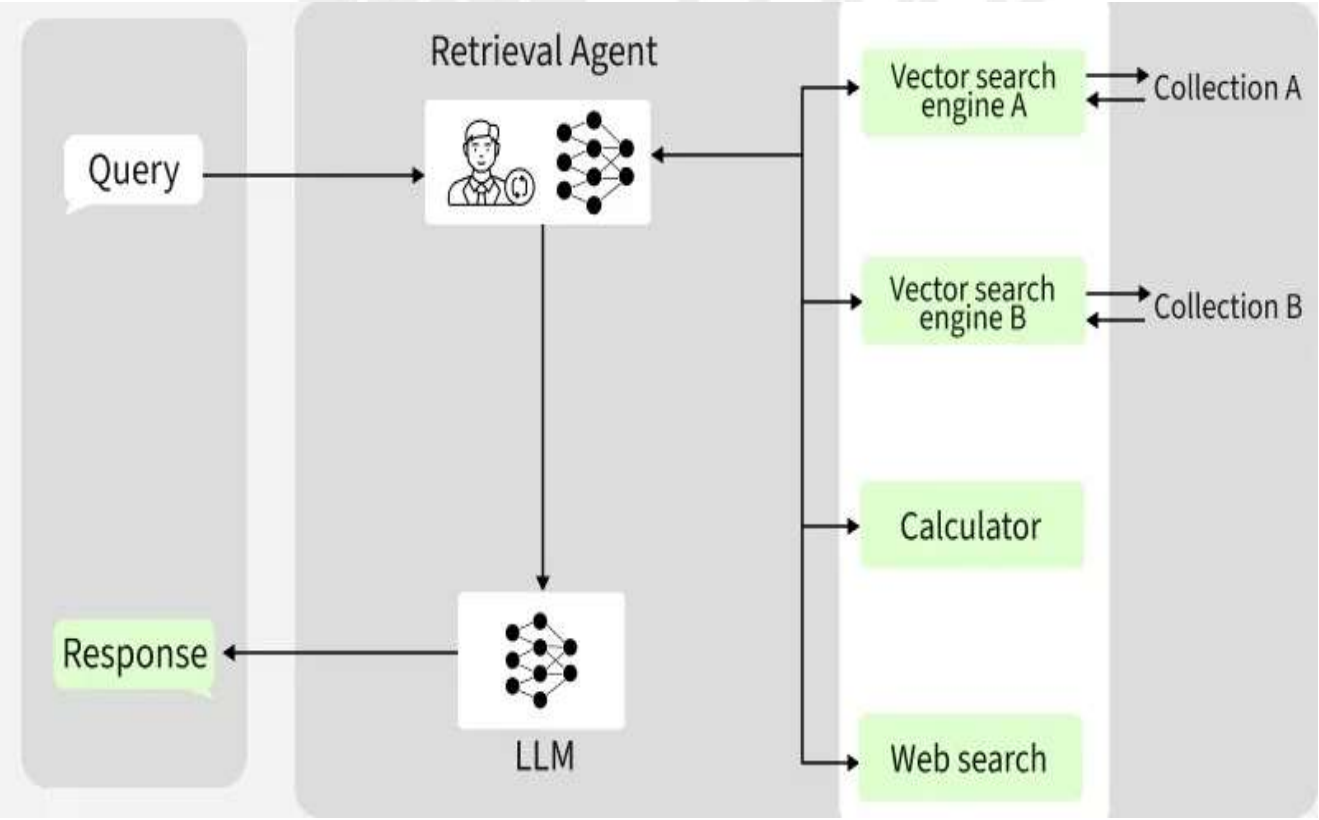
R.M.K GROUP OF INSTITUTIONS

2.1 Architecture of Agentic RAG

Agentic RAG architecture is designed to maximize adaptability and intelligence by leveraging autonomous agents and specialized tool integrations. At its core, the architecture organizes reasoning agents, each capable of decision-making, planning and retrieval, into a coordinated system.
Let's see key components of Agentic RAG Architecture,

1. Single-Agent RAG (Router)

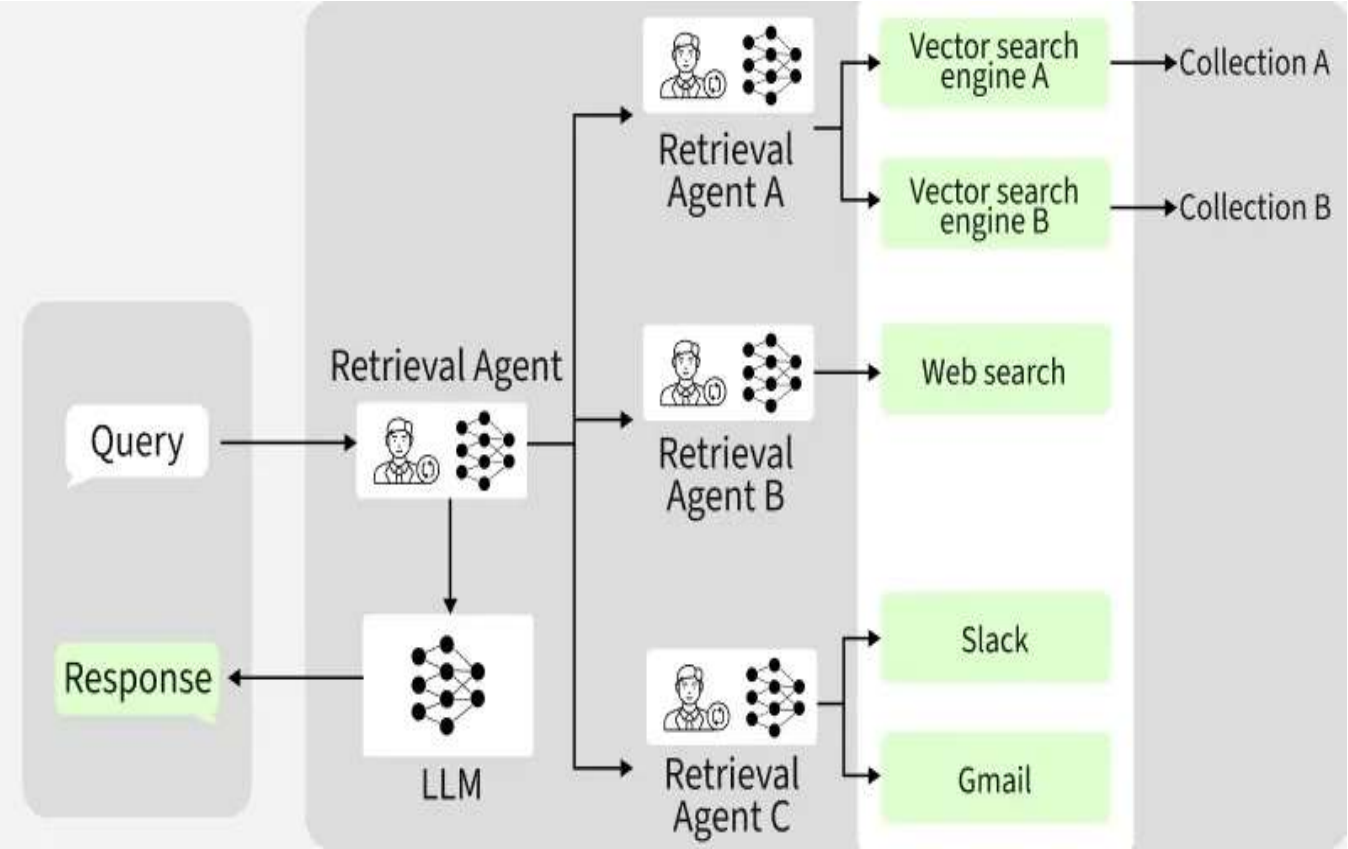
Single-Agent RAG uses a single intelligent agent that routes each user query to the most appropriate data source or tool. It excels in efficiently handling straightforward tasks without added complexity.



- Acts as a central dispatcher for query routing.
- Suitable for simple, well-defined questions.
- Chooses between fixed retrieval sources like search engines or databases.

2. Multi-Agent RAG

Multi-agent RAG involves a master agent coordinating multiple specialized sub-agents, each interacting with specific data sources or tools. It enables parallel processing of complex queries by dividing them into sub-tasks.



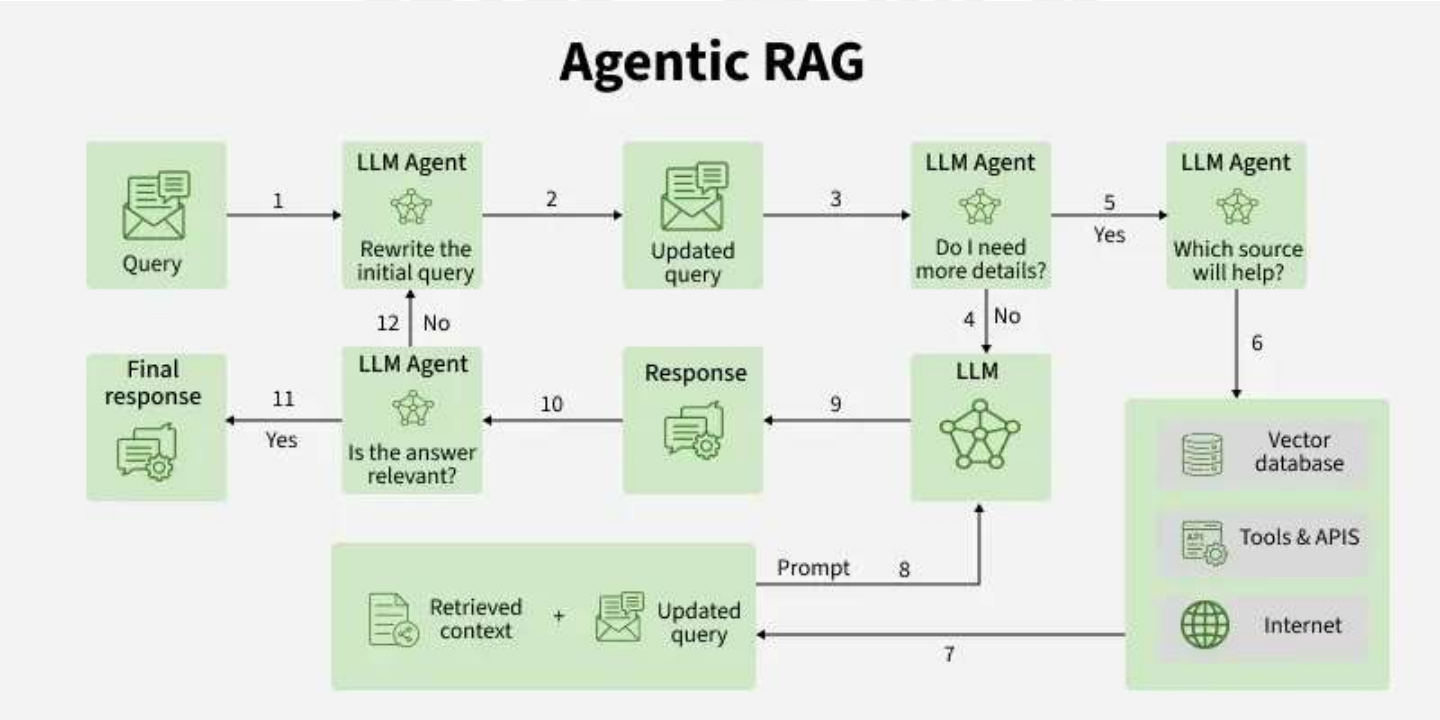
- Master agent supervises specialized sub-agents.
- Handles concurrent and parallel queries.
- Aggregates results for comprehensive, multi-source answers.

3. Agentic Orchestration

Agentic orchestration is the advanced coordination layer that lets agents dynamically plan, validate and iteratively refine workflows. It supports multi-modal data and adaptive strategy adjustment for richer, more accurate responses.

- Enables dynamic multi-step planning and feedback loops.
- Supports memory and intermediate result validation.
- Handles diverse data types like text, images and real-time inputs.

Working of Agentic RAG



- **Query Input**: The user submits a query, initiating the process.
- **Query Refinement**: An LLM agent reviews and rewrites the query for clarity, if needed, ensuring optimal data retrieval.
- **Information Sufficiency**: The agent checks if further details are needed. If so, more information is gathered before proceeding.
- **Source Selection**: The agent determines the best source for the query—vector database, APIs/tools or internet based on context.
- **Data Retrieval**: The chosen source is queried and relevant context is collected.
- **Context Integration**: Retrieved context is combined with the updated query to enrich understanding.
- **Response Generation**: The LLM produces a response using the enhanced context and query.
- **Answer Validation**: The agent verifies whether the response is relevant to the original question.
- **Final Output**: If validated, the system delivers a precise, context-aware final response.

R.M.K GROUP OF INSTITUTIONS

Components of Agentic RAG

1. Agent
The **agent** is the core decision-maker. It understands the user query, decides what actions to take, selects tools or data sources, and controls the overall workflow autonomously.

2. Planner
The **planner** breaks complex queries into smaller steps and decides the execution order. It enables multi-step reasoning and structured problem solving.

3. Retriever
The **retriever** fetches relevant information from external knowledge sources such as vector databases, documents, or repositories based on the agent's decision.

4. Knowledge Sources
These are external data stores like documents, databases, APIs, or web sources that provide factual and contextual information to reduce hallucinations.

5. Tool Interface
This component allows the agent to use external tools such as APIs, calculators, search engines, or code executors when required.

6. Memory
Memory stores past interactions and learned information.
Short-term memory: current conversation context
Long-term memory: user preferences and learned knowledge

7. Reasoning Engine (LLM)
The **LLM** performs reasoning, integrates retrieved context, and generates human-like responses.

R.M.K GROUP OF INSTITUTIONS

8. Orchestration Layer
Coordinates multiple agents and workflows, manages parallel tasks, and supports feedback loops for complex systems.

9. Validation Module
Checks accuracy and relevance of the generated response and refines it if necessary.

10. Response Generator
Produces the final, context-aware and validated answer for the user.

Variants of Agentic RAG

Agentic RAG can be implemented in different architectural styles depending on the complexity of the task. The main variants are **Single-Agent RAG, Multi-Agent RAG, and Agentic Orchestration**.

1. Single-Agent RAG

Single-Agent RAG uses **one intelligent agent** to manage the entire process such as understanding the query, retrieving information, and generating the response.

Working
•The agent receives the user query.
•It selects the suitable data source like database or API.
•Retrieves relevant information.
•Generates the final answer.

Features
•Centralized control
•Simple architecture
•Faster execution

R.M.K GROUP OF INSTITUTIONS

Use Cases
•FAQ systems
•Basic chatbots
•Simple document retrieval

2. Multi-Agent RAG

Multi-Agent RAG uses **multiple specialized agents** coordinated by a **master agent** to solve complex tasks.

Working
•Master agent receives the query.
•Breaks it into smaller tasks.
•Assigns tasks to different agents (retrieval, analysis, validation).
•Combines results to produce the final response.

Features
•Parallel processing
•Task specialization
•Handles multi-source data

Use Cases
•Research assistants
•Enterprise knowledge systems
•Healthcare and finance applications

R.M.K GROUP OF INSTITUTIONS

3. Agentic Orchestration

Agentic orchestration is an advanced variant where multiple agents are **dynamically coordinated** using planning and feedback.

Functions

- Controls workflow between agents
- Supports multi-step reasoning
- Validates intermediate results
- Adjusts strategy if needed

Features

- High flexibility
- Adaptive decision making
- Supports complex real-world problems

Use Cases

- Autonomous AI assistants
- Smart enterprise automation
- Industrial AI systems

R.M.K GROUP OF INSTITUTIONS

Applications of Agentic RAG

Agentic RAG is used in systems where AI must **retrieve information, reason, make decisions, and act autonomously**. It is suitable for applications that require **dynamic data and multi-step problem solving**.

1. Intelligent Chatbots and Virtual Assistants

Agentic RAG improves chatbots by allowing them to:
Retrieve real-time information
Use tools and APIs
Provide accurate and context-aware responses

Examples

Customer support bots
College enquiry assistants
Personal AI assistants

2. Enterprise Knowledge Management

Organizations use Agentic RAG to:
Retrieve data from internal documents
Answer employee queries
Summarize reports and policies

Benefits

Faster information access
Reduced manual search
Improved productivity

3. Healthcare Decision Support

Agentic RAG helps doctors by:
Retrieving patient history
Accessing medical guidelines
Providing treatment suggestions

Uses

Clinical decision support
Medical research assistance
Patient monitoring systems

R.M.K GROUP OF INSTITUTIONS

4. Financial and Business Analytics
In finance, Agentic RAG can:
Fetch real-time market data
Analyze trends
Generate insights and reports
Applications
Stock analysis
Risk assessment
Fraud detection support

5. Research and Education Assistants
Agentic RAG supports students and researchers by:
Retrieving research papers
Summarizing content
Generating explanations
Examples
Study assistants
Literature review tools
Academic recommendation systems

6. Recommendation Systems
Agentic RAG improves recommendations by:
Analyzing user preferences
Retrieving relevant data
Providing personalized suggestions
Examples
E-commerce product recommendations
Movie and music suggestions

7. Autonomous Workflow Automation
Used in organizations to automate:
Report generation
Email responses
Task scheduling
Data processing

R.M.K GROUP OF INSTITUTIONS

2.2.3 Agentic RAG with Llamaindex

We are building an Agentic RAG using LlamaIndex which is a system that allows an autonomous agent to retrieve relevant information from a set of documents and generate accurate responses. This system combines the retrieval capabilities of LlamaIndex with the reasoning and decision-making capabilities of agents.

Retrieval-Augmented Generation (RAG) has become one of the most effective techniques for building intelligent applications that can answer questions using external knowledge. However, traditional RAG systems only retrieve and return information—they do not reason, plan, or take actions.

This is where **Agentic RAG** comes in.

Agentic RAG combines retrieval with agent-like reasoning, allowing the Large Language Model (LLM) to not only fetch relevant information but also decide *how* to use it. This enables multi-step reasoning, self-correction, tool usage, and dynamic decision-making.

LlamaIndex is a powerful framework that simplifies the creation of Agentic RAG systems. It provides tools for:

Reading and processing PDFs

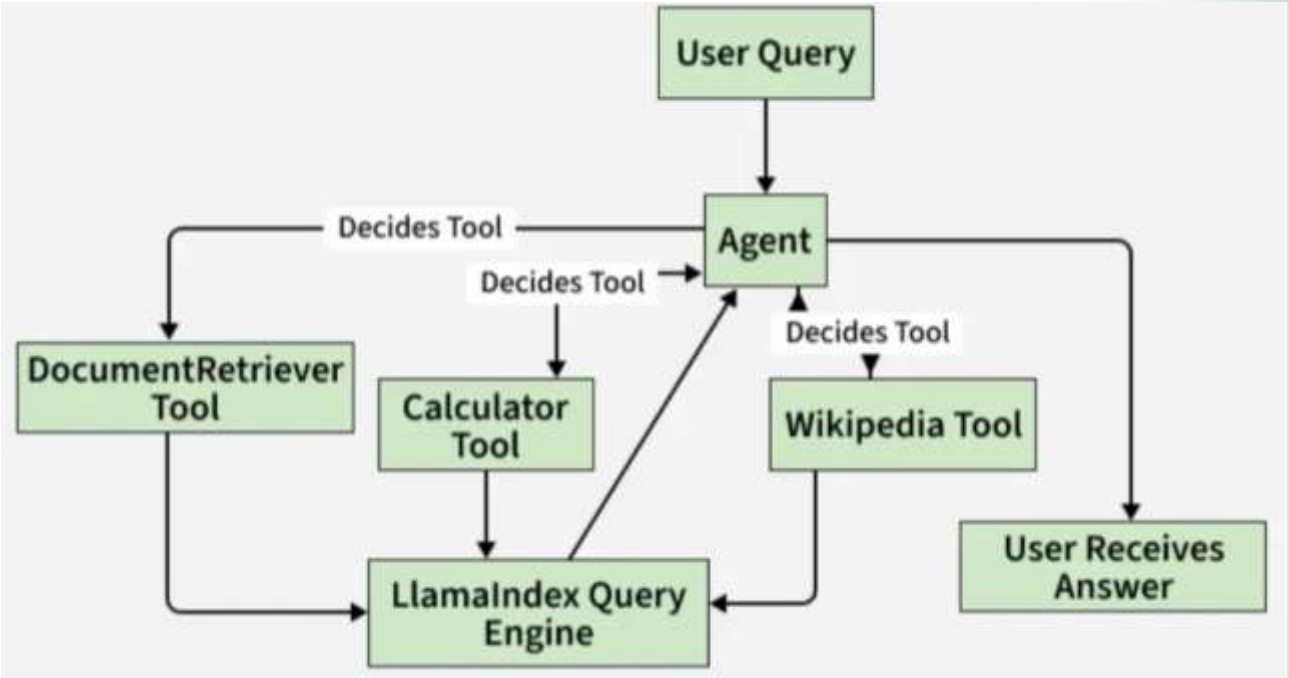
Building vector indexes

Retrieving relevant information

Creating LLM-powered agents that can use these tools intelligently

R.M.K GROUP OF INSTITUTIONS

Working of Agentic RAG System



Let's see how our system will be working:

1. User Query: Everything starts with a user question. The query flows into the central component i.e the Agent.

2. The Agent: The Agent acts as the "brain" of the system. Its job is to analyze the query and decide which specialized tool should handle different parts of the request. Instead of a fixed path, the agent makes dynamic decisions based on what the query needs.

3. Decision-Making and Tools: Depending on the query, the agent can choose between several tools:
DocumentRetriever Tool: Finds and fetches relevant documents for context.
Calculator Tool: Handles mathematical or computational questions.
Wikipedia Tool: Searches for factual knowledge directly from Wikipedia.
The agent can also call tools multiple timesor use a combination, depending on the task complexity.

4. **LlamaIndex Query Engine:** Some tools like DocumentRetriever or Calculator Tool, feed their results into the LlamaIndex Query Engine (a specialized search and synthesis engine). LlamaIndex processes and combines information from those tools to create a detailed and accurate answer.

5. Final Output: Once the agent is satisfied with the results, it sends the answer back to the user.

Step-by-Step Implementation
Let's build our Agentic RAG system which uses Llama-index:

Step 1: Install Dependencies
We will install the required packages and libraries for our system,
llama-index: For document retrieval and embeddings.
langchain: For agent and tool management.
langchain_community: Required for ChatOpenAI in LangChain 0.3.x.
openai: For LLM API access.
wikipedia: Optional tool for agent to search Wikipedia.

Step 2: Upload Documents and OpenAI API Key
We will upload some documents and files which our model can use. Files
Creates a docs/ folder to store our knowledge documents.
Users can upload .txt files.
Example content can include notes, articles or any text relevant to queries.

Step 3: Import Libraries
We will import the required libraries for system,
SimpleDirectoryReader: Load documents.
GPTVectorStoreIndex: Create vector-based index for retrieval.
LLMPredictor & ServiceContext: Wrap LLM for LlamaIndex.

R.M.K GROUP OF INSTITUTIONS

ChatOpenAI: OpenAI GPT model for text generation.
Tool, initialize_agent, AgentType: Build agentic reasoning system.
ConversationBufferMemory: Maintain past conversation context for agent.
wikipedia: Tool for retrieving general knowledge.

Step 4: Build the LlamaIndex Retrieval System
We will build the LlamaIndex Retrieval System in which,

SimpleDirectoryReader: Reads all uploaded documents.
LLMPredictor: Wraps GPT-3.5-turbo to work with LlamaIndex.
GPTVectorStoreIndex: Converts documents into embeddings stored in a vector store.
query_engine: Returns top 3 most relevant documents for any query.

Step 5: Define Tools for Agent
We will define the tools that are available for the agent to use:
DocumentRetriever: Uses LlamaIndex to fetch relevant docs.
Calculator: Handles numeric queries.
Wikipedia: Fetches general knowledge not in uploaded docs.
Tools: Each tool is callable by the agent automatically.

Step 6: Initialize Agent with Memory
We will initialize ConversationBufferMemory which is a short-term, in-session memory to the agent:
ConversationBufferMemory: Keeps track of past queries and responses.
AgentType.ZERO_SHOT_REACT_DESCRIPTION: Agent decides which tool to call without pre-training.
verbose=True: Shows reasoning steps in the output.

R.M.K GROUP OF INSTITUTIONS

Step 7: Run the System

We run our system in which:

Users can interact with the agent.

Agent decides which tool to use, retrieves relevant info and generates answers.

Supports document queries, math calculations and Wikipedia search.

Advantages

- **Autonomous Reasoning:** The agent intelligently decides which tool to use for each query without manual intervention.
- **Accurate Responses:** LlamaIndex retrieves only the most relevant document chunks before generating answers, reducing hallucinations.
- **Multi-Tool Support:** Supports multiple integrated tools such as PDF retrieval, calculators, search engines, and Wikipedia queries.
- **Context-Aware:** Maintains conversation memory, allowing the user to ask follow-up or clarification questions naturally.
- **Scalable & Modular:** New tools, documents, and data sources can be added or updated easily as the system grows.
- **User-Friendly:** Provides natural, interactive language responses suitable for both technical and non-technical users.
- **Improved Efficiency:** Reduces manual document searching, speeding up information access.
- **Better Decision-Making:** Agentic reasoning enables multi-step thinking and structured problem solving.

Disadvantages

Dependency on LLM Quality: The accuracy heavily depends on the performance of the underlying language model.

•**Computational Cost:** Using agents and embeddings requires significant compute power, especially for large documents.
•**Latency Issues:** Multi-step reasoning can increase response time compared to simple chat models.
•**Embedding Limitations:** Incorrect or poor-quality embeddings can lead to irrelevant or incomplete retrieval.
•**Complex Setup:** Requires initial configuration of indexes, tools, and integrations, which may be challenging for beginners

2.2.4 Agentic RAG with Cohere

Agentic Retrieval-Augmented Generation (Agentic RAG) is an advanced version of RAG where the model not only retrieves documents but also behaves like an **agent**—capable of reasoning, planning, and deciding which tools to use.
By integrating **Cohere** LLMs with **LlamaIndex**, we can create a powerful Agentic RAG pipeline that:

- Reads PDFs or external documents
- Retrieves relevant chunks
- Uses Cohere for reasoning
- Allows multi-step tool-driven thinking

Produces more accurate and grounded responses
Cohere's language models (like **Cohere Command**, **Command-R**, etc.) are strong at retrieval, grounding, and reasoning — making them ideal for Agentic RAG applications.

How Agentic RAG Works with Cohere:

1. Data ingestion
Load PDFs or text documents
Split and embed text using embedding models (Cohere or HuggingFace)
2. Index creation
Store embeddings in a vector store
Build a query engine

R.M.K GROUP OF INSTITUTIONS

3. Agent Creation

LLM (Cohere Command model) becomes the reasoning engine

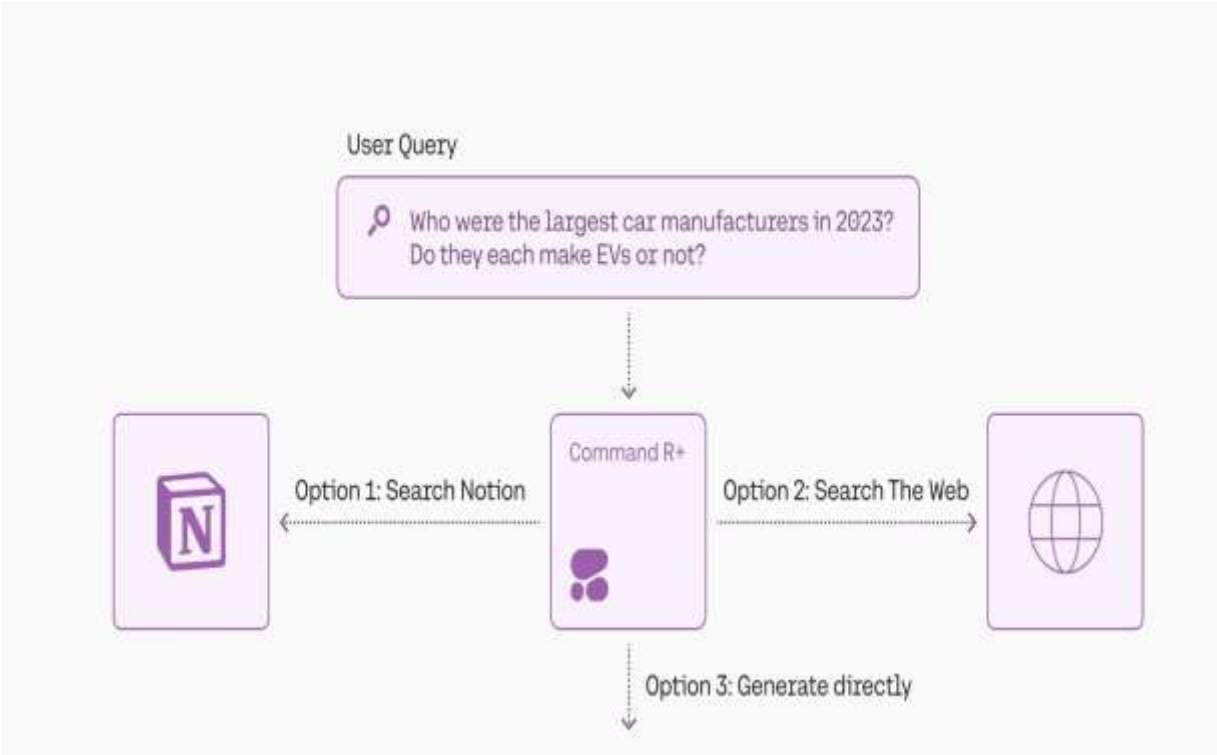
Tools (RAG query engine, calculator, Wikipedia API, etc.) are attached

The agent decides which tool to call using autonomous reasoning

4. Query Execution

The agent:understands the question retrieves relevant information processes it gives a final answer

Building Agentic RAG with Cohere



1. User Query

A user provides a question — for example:

“Who were the largest car manufacturers in 2023? Do they each make EVs or not?”

This query requires up-to-date, factual information. The agent must determine where to retrieve the most relevant data.

2. Command R+ (Agent Decision Layer)

The query is passed to **Command R+**, an LLM that performs:

Reasoning

Planning

Tool selection

Source evaluation

The model decides the *best method* to answer the question.

3. Three Possible Actions (Tool Choices)

Command R+ can choose from three different retrieval/generation pathways depending on the query:

Option 1: Search Notion

If the organization stores internal knowledge or documentation in Notion, the agent may:

Retrieve records about companies

Fetch internal reports on EV production

Look up company profiles stored in team workspaces

Use Case: When the answer exists in internal knowledge bases

Option 2: Search the Web

If the question requires **current external information**, such as updated manufacturing rankings or EV production stats, the agent may perform a web search.

The system can gather:

Latest statistics

Market reports

.

R.M.K GROUP OF INSTITUTIONS

News articles
Industry rankings
Use Case: When the information is not available internally or needs real-time updates.

Option 3: Generate Directly
If the answer:
Does not require external sources
Is general knowledge
Or is a creative/explanatory query
Then the agent may simply generate the response using its internal knowledge.
Use Case: When reasoning or explanation is enough without retrieval.

Advantages
IntelligentToolSelection:
The agent (Command R+) automatically decides whether to search internal documents, search the web, or generate directly — reducing manual effort.
Higher Accuracy:
By retrieving verified information from sources like Notion or the web, answers become more factual and grounded.
Up-to-Date Responses:
Web search enables the system to answer questions that require the latest information, such as market data or recent news.
Flexible Information Sources:
The system can use multiple knowledge bases (Notion, web, external datasets) depending on the query.
Reduced Hallucinations:
Retrieval-based answers reduce the chance of the model producing incorrect or invented information.

R.M.K GROUP OF INSTITUTIONS

Better User Experience:

Users receive more accurate, personalized, and context-aware responses, even for complex multi-part questions.

Scalable Architecture:

New tools, APIs, or data sources can be added easily as the system grows.

Disadvantage:

Complex Setup:

Requires configuring multiple tools (Notion, web search, local retrieval), which can be challenging for beginners.

Higher Latency:

When the agent searches the web or internal databases, responses may take longer than simple generation.

Tool Dependency:

If an external tool (e.g., web search API or Notion API) fails, the system may produce incomplete answers.

Risk of Outdated Internal Data:

If Notion or internal sources are not updated, the system may return outdated information.

R.M.K GROUP OF INSTITUTIONS

Content beyond Syllabus

Concept

In advanced Agentic RAG systems, the agent does more than simply retrieve documents. Instead of blindly trusting whatever is retrieved, the agent **evaluates, critiques, and filters** the information before using it. This makes the final output more reliable and grounded.

How It Works

Agent retrieves documents from the vector database or external source.

Agent evaluates the retrieved content for accuracy, relevance, and consistency.

Low-quality or conflicting documents are rejected, preventing incorrect context from influencing the answer.

Only validated and trustworthy knowledge is used for response generation.

Why This Matters

Reduces hallucinations

Improves factual correctness

Enables explainable reasoning

Tool-Aware Agentic RAG (Dynamic Tool Selection)

Beyond Static Pipelines

Instead of fixed retrieval → generation flow, the agent decides:

When retrieval is needed

Which data source to query

Whether to use search, DB, API, or calculator

Example

If retrieval confidence is low:

Agent triggers external web search

Or asks clarifying questions

R.M.K GROUP OF INSTITUTIONS

Quiz

1. What best describes Agentic RAG?

Retrieval system without reasoning

b) A static pipeline that only fetches documents

c) A system where an agent actively retrieves, evaluates, and uses information to solve tasks

d) A model that generates text without external data

Answer: c

2. How does Agentic RAG differ from Traditional RAG?

a) Agentic RAG uses smaller models

b) Agentic RAG retrieves more documents

c) Agentic RAG can make decisions, reflect, and use tools dynamically

d) Traditional RAG is faster

Answer: c

3. Which component is responsible for tool selection and reasoning in Agentic RAG architecture?

Retriever

b) Knowledge Base

c) Agent Controller / Orchestrator

d) Embedding Model

Answer: c

4. Which variant of Agentic RAG focuses on evaluating the quality of retrieved documents before use?

a) Multi-Agent RAG

b) Self-Evaluating Agentic RAG

c) Tool-Augmented RAG

d) Basic RAG

Answer: b

R.M.K GROUP OF INSTITUTIONS

Assignments

1.Explain the role of the following components:

- Retriever
- Agent Controller
- Knowledge Base
- Reasoning Module

2. Describe how tool selection works in an Agentic RAG pipeline.

3. With a neat diagram, explain the architecture of a Self-Evaluating Agentic RAG system

4.Define Agentic RAG in your own words. How is it different from a normal RAG system?

5. Explain the need for autonomous reasoning in modern retrieval-augmented systems.

R.M.K GROUP OF INSTITUTIONS

Mini Project

Q. No.	Question	CO Level	K Level
1	Develop an autonomous agent that solves a real-world problem using the ReAct (Reasoning and Acting) pattern.	CO2	K3
2	Design a multi-agent system where multiple agents collaborate to achieve a common goal.	CO2	K3
3	Design an Agentic AI system that reflects on its own decisions and ensures secure and ethical behavior.	CO2	K3

Real Time Applications

1. Customer Support Autonomous Agents
2. Healthcare Monitoring & Care Agents
3. Autonomous Trading & Finance Agents
4. Smart Manufacturing Agents

R.M.K
GROUP OF
INSTITUTIONS

Part A

1. What is Agentic RAG?

Agentic RAG is Retrieval-Augmented Generation where the model autonomously plans, reasons, and decides which tools or data sources to use instead of following a fixed retrieval pipeline.

2. **How does Agentic RAG differ from Traditional RAG?**

Traditional RAG follows a fixed “retrieve → generate” flow, while Agentic RAG dynamically decides when and what to retrieve and can use tools, memory and reasoning loops.

3. What is the main limitation of Traditional RAG?

Answer:

It cannot verify or critique retrieved documents, leading to hallucinations when retrieval is irrelevant or insufficient.

4.What is the key advantage of Agentic RAG?

Answer:

It performs autonomous decision-making, improving accuracy and reducing hallucinations.

5.What are the core components of Agentic RAG architecture?

Answer:

- Reasoning module
- Retrieval module
- Tools/APIs
- Memory
- Generator LLM

R.M.K GROUP OF INSTITUTIONS

6. **What is the Reasoning Loop?**
The reasoning loop is the core of Agentic RAG that enables planning and self-correction.
The agent analyzes the query, forms a plan, retrieves data, critiques it, and acts again if required.
This loop continues until the answer is accurate.
It makes the agent autonomous and adaptive
.

7.Describe the Tool Execution Layer.
This layer allows the agent to use external tools like search APIs, databases, calculators, and more.
It helps the agent fetch updated or missing information.
Tool usage is determined dynamically based on the

8. What is Self-Evaluating Agentic RAG?
In this variant, the agent critiques retrieved documents before using them.
It checks accuracy, relevance, and consistency.
If documents are poor-quality, they are rejected.

9. What is Tool-Aware Agentic RAG?
Tool-Aware RAG allows the agent to decide which tool to use and when.
It may choose between internal retrieval, web search, APIs, or calculators.
The decision depends on context and confidence.

10. What is Multi-Agent RAG?
Multi-Agent RAG uses multiple agents that collaborate, each with its own role.
One may retrieve documents, another evaluates them, and another generates responses.
This division of tasks increases accuracy.
It is often used in research or enterprise systems.

.

R.M.K GROUP OF INSTITUTIONS

11Explain the Reflection Pattern in Agentic RAG.

The reflection pattern allows the agent to critique its own output.
After generating a draft answer, the agent reviews it for mistakes.
It then provides a corrected and improved version.
This self-refinement enhances answer quality.

12._What are the applications of Agentic RAG?

Agentic RAG is used in enterprise search, research assistants, automation, and customer support.
It helps in legal, medical, engineering, and academic domains.
It can autonomously analyze documents and retrieve verified answers.
Its reliability makes it suitable for high-stakes environments.

13. What is Agentic RAG with LlamaIndex?

LlamaIndex provides indexing, retrieval, agents, and tool integrations.
It supports building autonomous systems that plan and act.
Tools like SQL, web search, and PDF reading are built-in.

14. Why is Agentic RAG useful in enterprises?

Enterprises have large unstructured knowledge bases.
Agentic RAG can autonomously navigate complex information, validate it, and produce accurate insights.
It also supports workflow automation using tools.
This increases productivity and reduces manual research time.

15. How does LlamaIndex enable tool use?
LlamaIndex includes a ToolRegistry and AgentRunner.
Agents can pick tools dynamically based on the query.
Tools can include database queries, document lookup, or API calls.
This enables complex autonomous workflows.

16. What is Agentic RAG with Cohere?
Cohere's Command R+ model supports tool calling and web search.
It can evaluate whether a query needs retrieval, generation, or external search.
This makes responses more accurate and up-to-date.
It is widely used for production-grade RAG systems.

17. How does Cohere handle retrieval?
Cohere can search vector databases, websites, and enterprise systems.
It decides autonomously whether retrieval is needed.
It can combine multiple data sources for better context.
This ensures factual and comprehensive answers.

18. Why is Cohere suitable for enterprise RAG?
Cohere provides secure APIs, governance controls, and high-quality embeddings.
It supports multi-document reasoning and tool execution.
Enterprises benefit from its reliability and speed.
It also integrates easily with cloud platforms.

R.M.K GROUP OF INSTITUTIONS

19. What are the advantages of Agentic RAG?

It reduces hallucinations by verifying retrieved data.
It has autonomous reasoning and tool usage capabilities.
It can adapt to complex, multi-step queries.
The system is scalable, modular, and explainable

20. What are the limitations of Agentic RAG?

It requires more computational resources than basic RAG.
Tool integrations must be carefully designed.
Poorly configured retrieval or tools may reduce performance.
Despite this, its benefits usually outweigh the limitations

R.M.K GROUP OF INSTITUTIONS

Part B

1. Explain the complete architecture of Agentic RAG with a neat diagram.

2. Compare Agentic RAG and Traditional RAG in terms of workflow, accuracy, limitations, and decision-making abilities.

3. Describe the reasoning loop in Agentic RAG and explain how it reduces hallucinations.

4. Discuss the role of tools in Agentic RAG. How does dynamic tool selection improve system performance?

5. Explain Self-Evaluating Agentic RAG. How does document filtering and critique improve retrieval quality?

6. Elaborate on the different variants of Agentic RAG with suitable examples.

7. Explain how Agentic RAG is implemented using LlamaIndex, including indexing, agents, and tools.

8. Describe the features of Cohere's Command R+ model and explain its role in Agentic RAG.

9. Discuss the major applications of Agentic RAG in enterprise systems and knowledge automation.

10. Explain the challenges, advantages, and limitations of building a real-world Agentic RAG pipeline.

R.M.K GROUP OF INSTITUTIONS

Supportive Online Courses

S.No	Courses	Platform
1.	Generative and Agentic AI	Coursera
2.	Agentic AI and AI Agents for Leaders	Coursera
3.	Agentic AI: Mastering AI Agents and Systems	Udemy
4.	Introduction to AI & Generative AI	Infosys Springboard
5.	Agentic AI: Mastering AI Agents and Systems	Udemy
6.	Intro to AI Agents & Agentic AI	Udemy

Text & Reference Books

Sl. No.	Book Name & Author	Book
1	1. 1. Anthony Chang, Designing Agentic AI Systems: Foundations, LLMs, and Multi-Agent Frameworks, Packt Publishing 2024. 2. 2. Richard M. Reese, Building LLM Applications with LangChain and Generative AI, O'Reilly Media, 2024.	Textbook
2	Thuyan Nguyen & Carlos Fernandez, Practical Multi-Agent Systems with CrewAI and AutoGen, Apress, 2024	Reference Book
3	Nachiappan, AI Research TeamComputer Generative AI with Python and LLMs, 6th edition, Morgan Kaufmann, 2021.	Reference Book
4	Mark Roy & Google Cloud Experts, Building AI Applications with Amazon Bedrock & Vertex AI, Wiley,Edition: 2024	Reference Book

INSTITUTIONS

Assessment Schedule

S. No	Assessment	Proposed Date	Actual Date
1	Unit Test I	06.01.2025	
2	Internal Assessment Test I	29.01.2025	29.01.2025
3	Unit Test II	25.02.2025	
4	Internal Assessment Test II	10.03.2025	10.03.2025
5	Model Examination	08-04-2025	08-04-2025

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K GROUP OF INSTITUTIONS